

SDK Hands-on Workshop

Nordic / u-blox / Avnet

Chapter 2: Bluetooth Low Energy

June 2026

nRF54L15-DK / EVK-NORA-B2

nRF Connect SDK v3.3.0



NORDIC[®]
SEMICONDUCTOR

Agenda

Chapter 2

- Introduction to Bluetooth Low Energy
- Adding Bluetooth Advertising to your project
- Building the `peripheral_lbs` sample
- Understanding the source code and modifying it

Bluetooth Low Energy



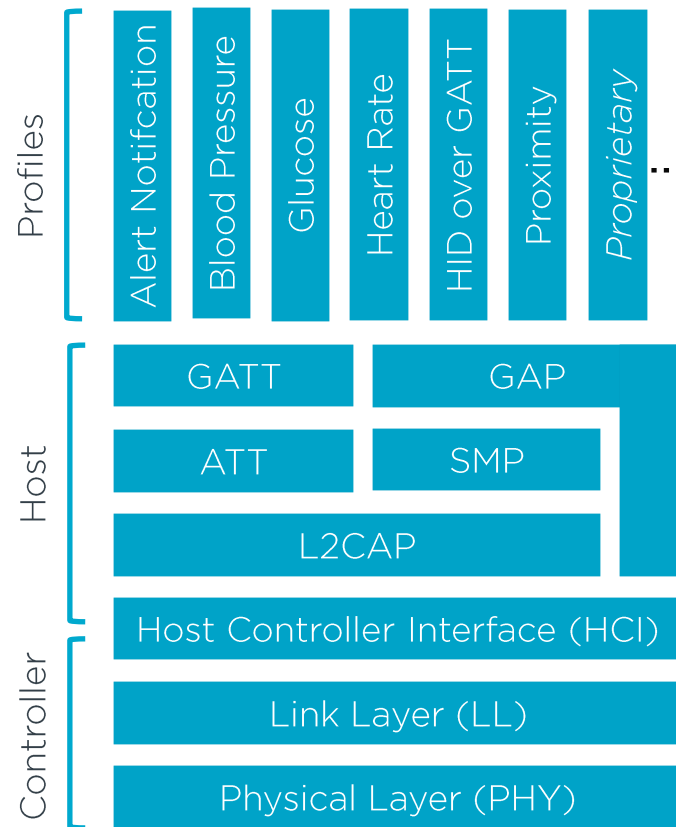
Bluetooth SIG



- Bluetooth Special Interest Group
- Develop and license Bluetooth Low Energy technology
- Network of member organizations
- Founded in September 1998
- 36000 member companies
- 5.4 billion Bluetooth product shipments in 2025
- Nordic is involved with BTLE since 2007 & joined the SIG in 2011
 - Involved in several working groups, driving the specifications

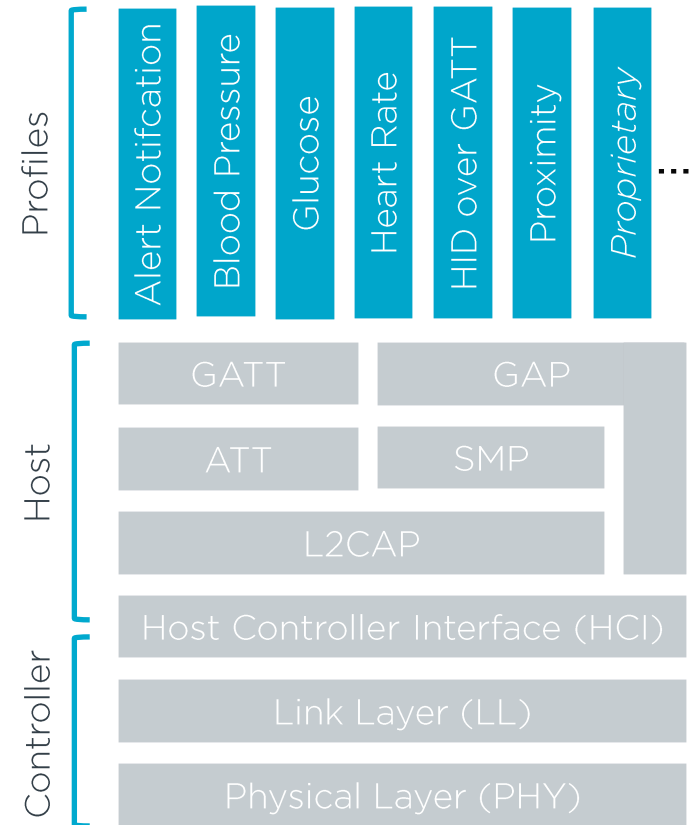
Architecture

- Controller – Radio connection
- Host – Connection, discovery, advertising
- Profiles – Services and characteristics



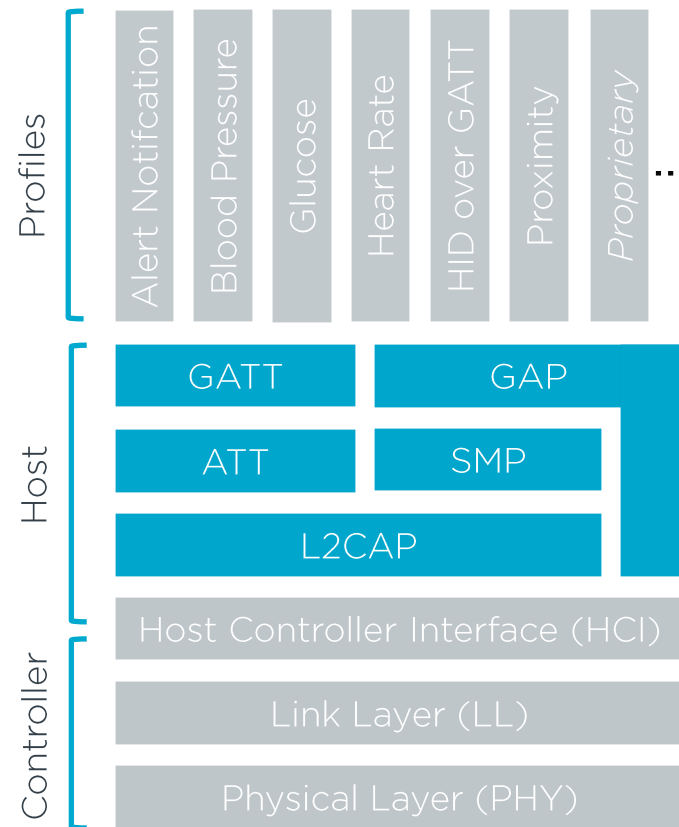
Profiles

- Profiles
 - Describes how two or more devices can discover and communicate with each other
 - Implements a specific application
 - Standard or proprietary
 - Each profile has its own specification



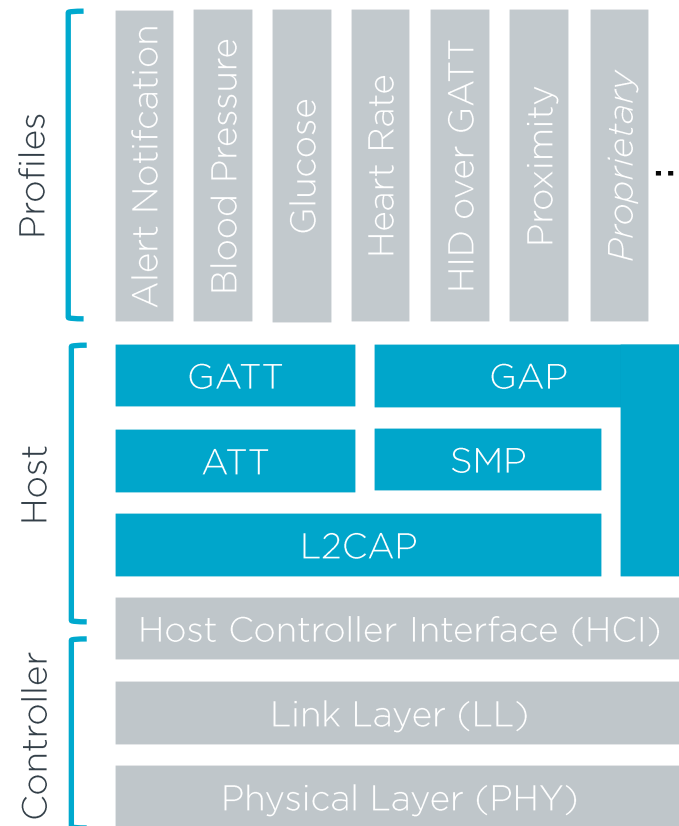
Host

- Upper layers of the Bluetooth LE protocol stack
- Logical Link Control and Adaption Protocol
- Attribute Protocol (ATT)
 - Simple client-server model
 - Client device can access attributes on the server device
- Security manager Protocol (SMP)
 - Defines protocol for pairing and key distribution



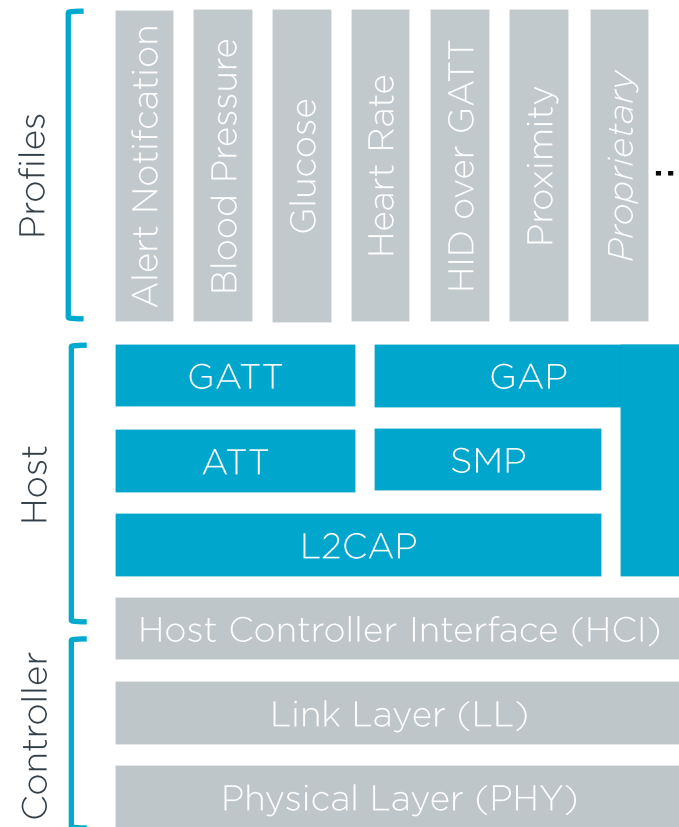
Host - GATT

- Generic Attribute Profile (GATT)
- Highest data layer
- Uses ATT to discover and access attributes
- Specifies a hierarchical structure of attributes
 - Services
 - Characteristics
 - Descriptors



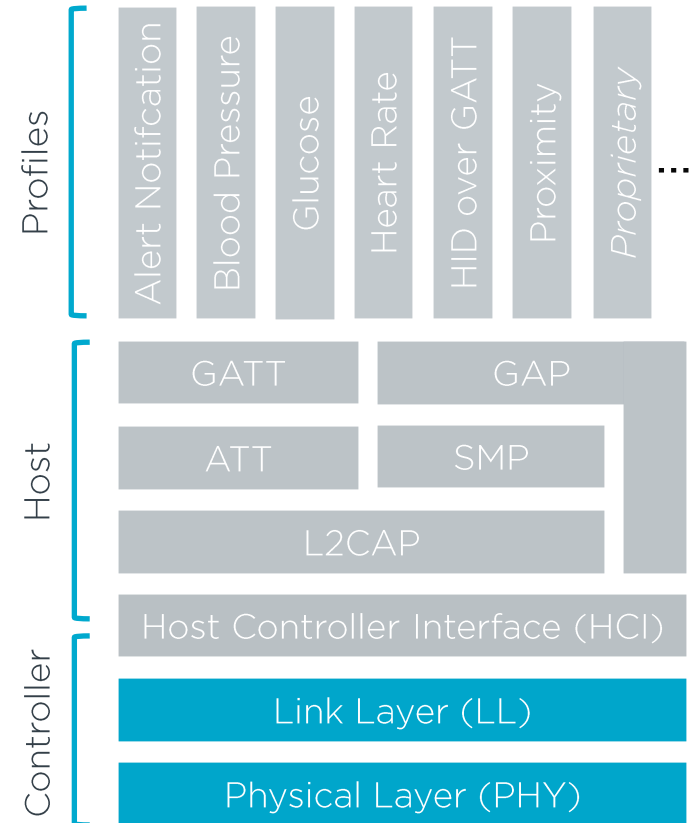
Host - GAP

- Generic Access Profile (GAP)
 - Highest control layer
 - Defines device roles
 - Defines how devices discover and connect to each other
 - Defines security modes and procedures



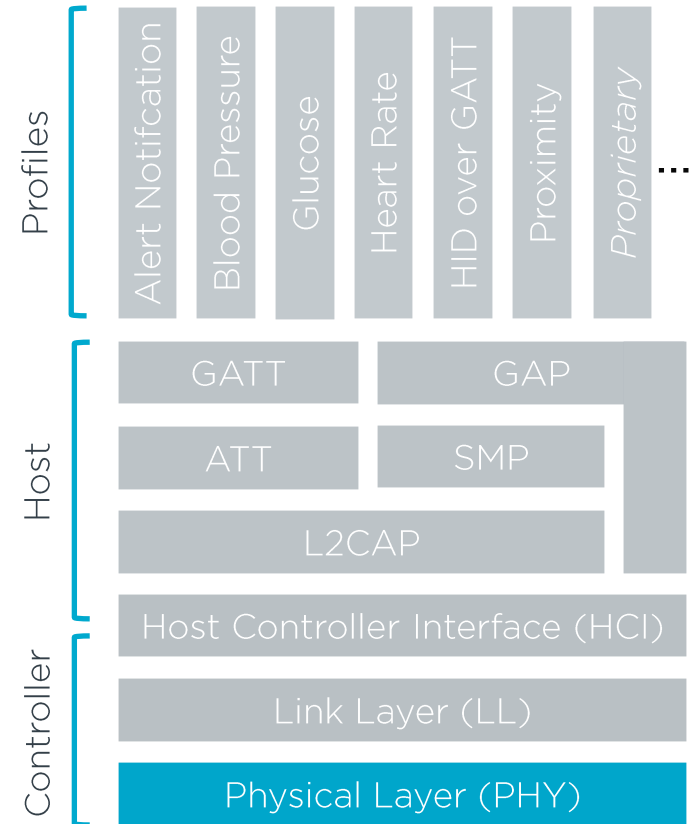
Controller

- Physical layer
 - Defines how two radios can send bits to each other
- Link Layer
 - Defines Link Layer states
 - Defines device address
 - Packet format



Physical layer

- 2.4 GHz ISM band
- 40 RF channels (2 MHz)
- GFSK modulation
 - 1 Mbps, 2 Mbps and other rates
- Max. 20 dBm TX power
 - In Europe restricted to 10 dBm



Physical layer – Radio Modes

- **1M PHY**
 - Classis PHY Supported by all BLE devices (default mode)
- **2M PHY**
 - Introduced in v5.0
 - Doubles the data rate, lowers power consumption.
 - Decreased sensitivity with translates to less range (about 80%)
- **Coded PHY**
 - Uses more symbols to send one bit
 - 2 different modes S2 500kbps (1Mbps/2) and S8 125kbps (1Mbps/8)
 - Longer transmission times resulting in longer range (about 4x)

GAP roles and Link Layer states

GAP role	Link Layer state
Broadcaster	Advertising
Observer	Scanning
Peripheral	Advertising Connection (Slave)
Central	Scanning Initiating Connection (Master)

All roles can also be in the standby state

Roles (GAP) - Example



- B Broadcaster
- O Observer
- P Peripheral
- C Central

Bluetooth Low Energy Added to your project



Bluetooth Advertising

- Advertising is the simplest way of using the BTLE stack
- It can be added with just a few modifications to the project

1. Add BTLE stack to your prj.conf
2. Add Bluetooth file headers
3. Construct the advertising packet
4. Initialize the BTLE stack
5. Start advertising in your main routine

```
CONFIG_BT=y
```

```
#include <zephyr/bluetooth/bluetooth.h>
```

```
static const struct bt_data ad[]
```

```
bt_enable(NULL);
```

```
bt_le_adv_start(BT_LE_ADV_NCONN, ad, ARRAY_SIZE(ad), NULL, 0);
```

Advertising Packet (legacy)

- Array with type struct `bt_data` which will be used to send out data. It consist of the advertising flags and the advertising packet data.

BT_DATA_BYTES() Is a macro to generate the AdvData flags

- **BT_DATA_FLAGS** Indicator of package type
- **BT_LE_AD_GENERAL** Indicate that advertising is available for long time (no time out)
- **BT_LE_AD_NO_BREDR** Classic Bluetooth not supported

BT_DATA() Is a macro to generate the AdvData payload

- **BT_DATA_NAME_COMPLETE** Indicator of package type
- **DEVICE_NAME** Advertising device name (max. 26 bytes/chars)
- **DEVICE_NAME_LEN** Length

Advertisement PDU



Payload

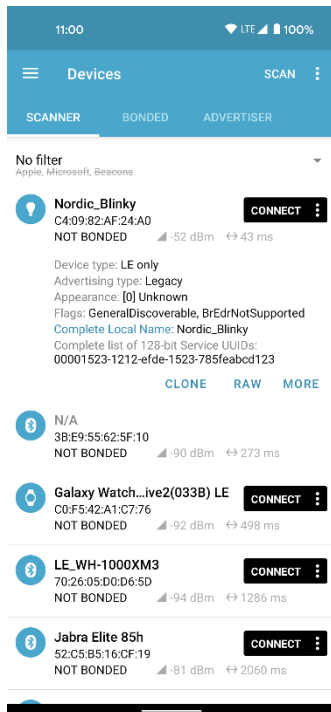


```

43 static const struct bt_data ad[] = {
44     BT_DATA_BYTES(BT_DATA_FLAGS, (BT_LE_AD_GENERAL | BT_LE_AD_NO_BREDR)),
45     BT_DATA(BT_DATA_NAME_COMPLETE, DEVICE_NAME, DEVICE_NAME_LEN),
46 };

```

nRF Connect for Mobile



Scan for your device using the nRF Connect for Mobile app

You can use RSSI thresholds/filters to quickly identify your device in a list of many.



Nordic LBS Sample

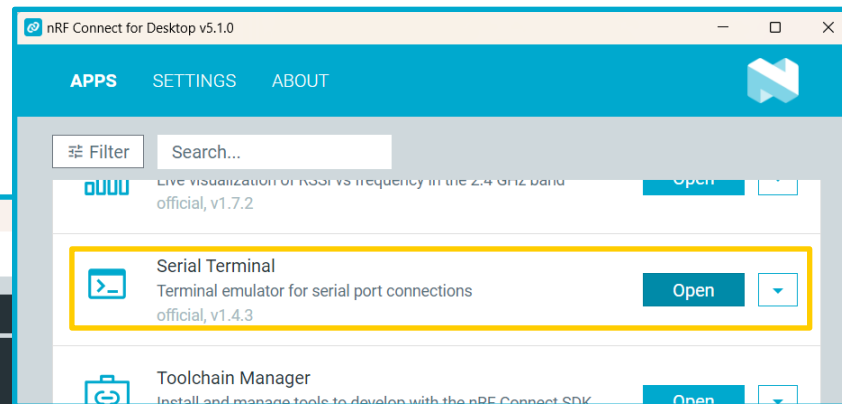
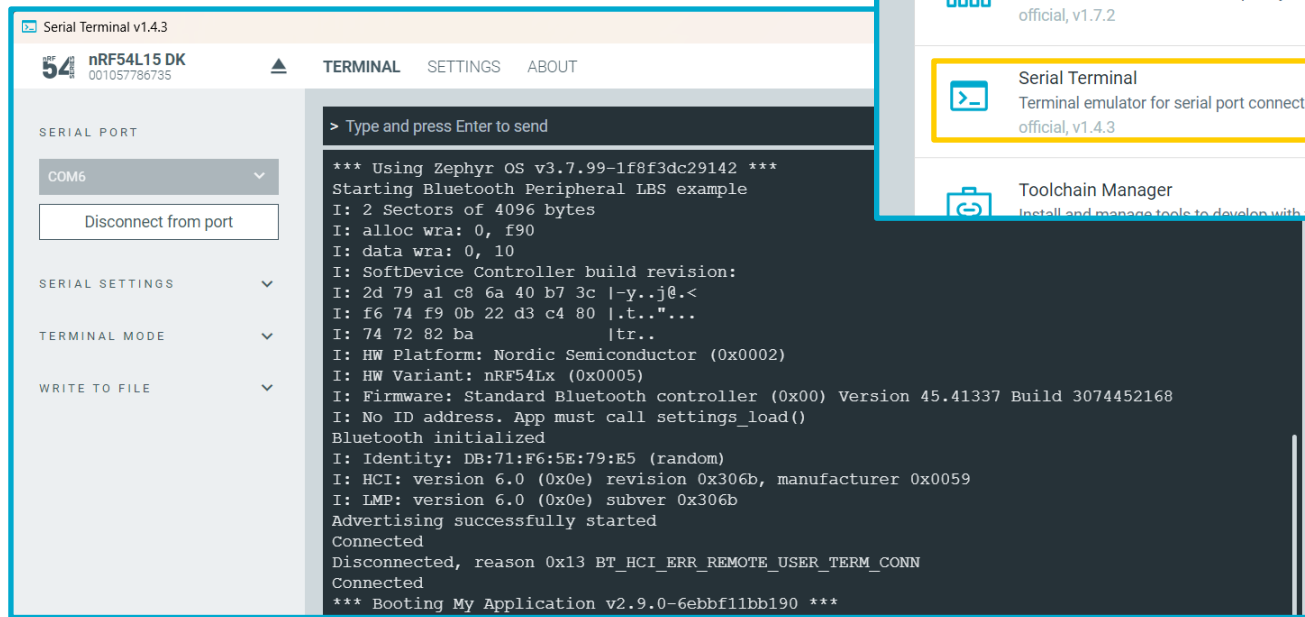
LED and Button Service (LBS)

Nordic Led/Button Service

The Blinky with Bluetooth Low Energy

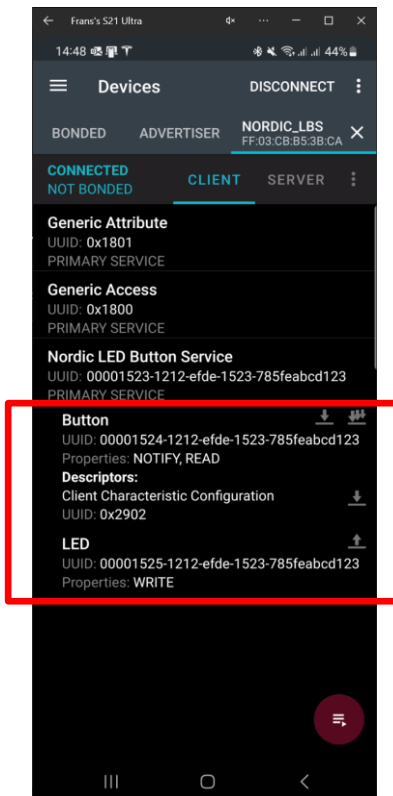
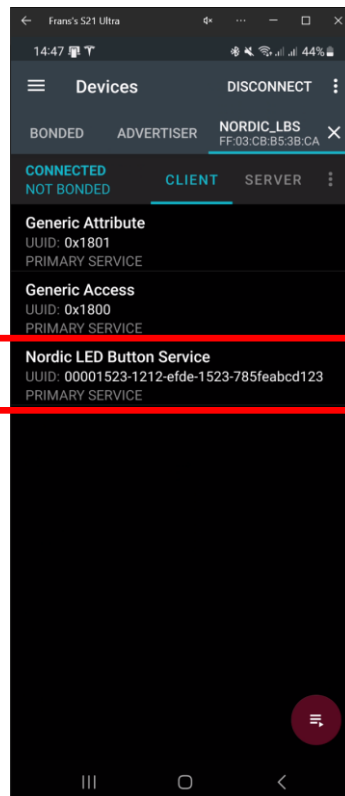
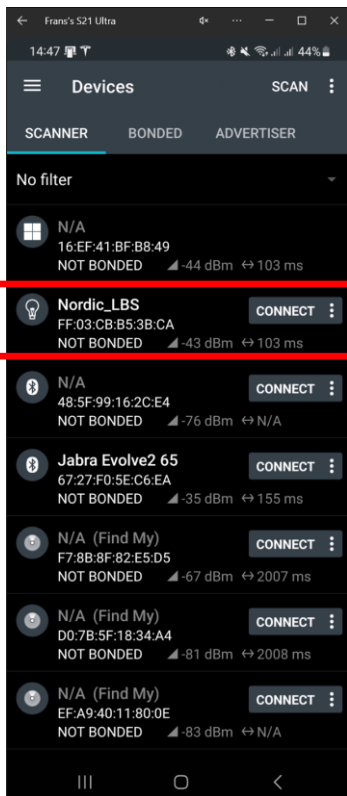
- Build the Nordic “peripheral_lbs” sample
- Test application using the nRF Connect Mobile app
- Change the BLE advertising type
- Add handler for connected state

nRF Connect For Desktop Serial Terminal

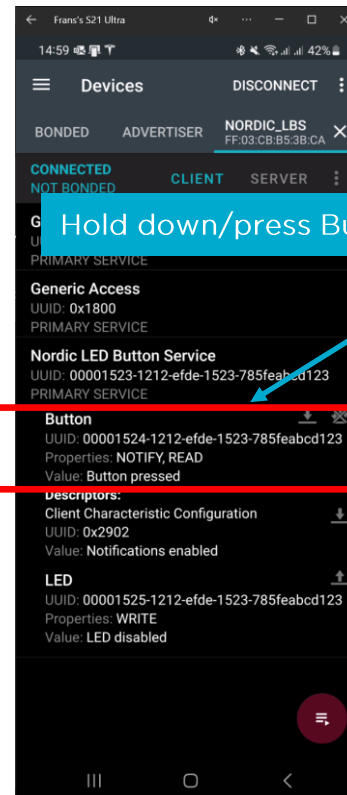
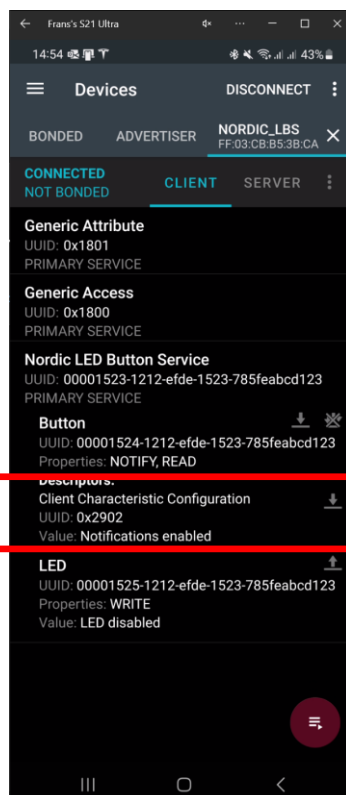
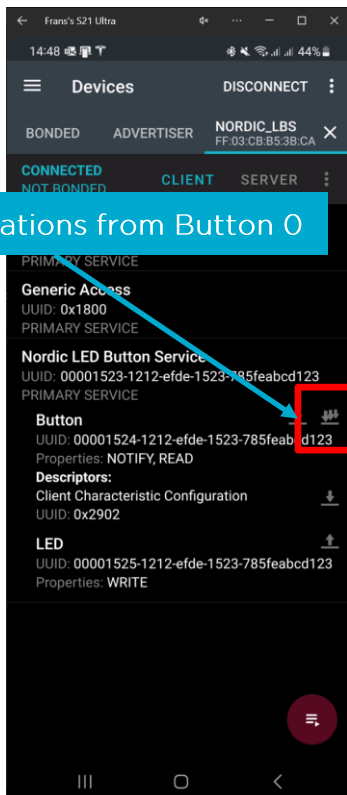


- **LED 0:**
Blinks when the main loop is running (that is, the device is advertising) with a period of two seconds, duty cycle 50%.
- **LED 1:**
Lit when the development kit is **connected**.
- **LED 2:**
Can be activated/deactivated through a Bluetooth write operation.
- **Button 0:**
Send a notification with the button state: "pressed" or "released".

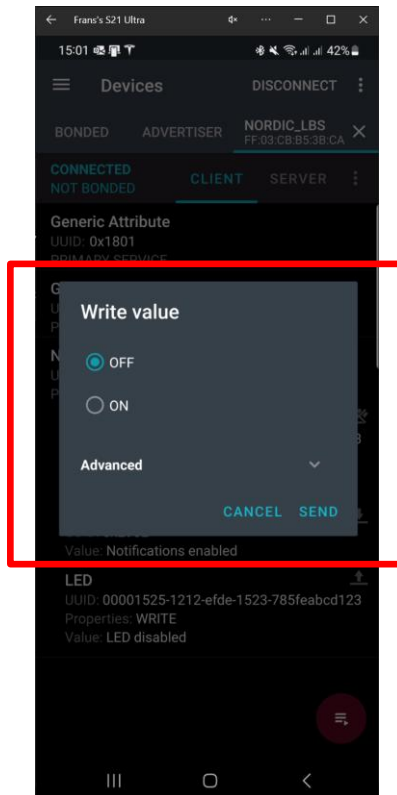
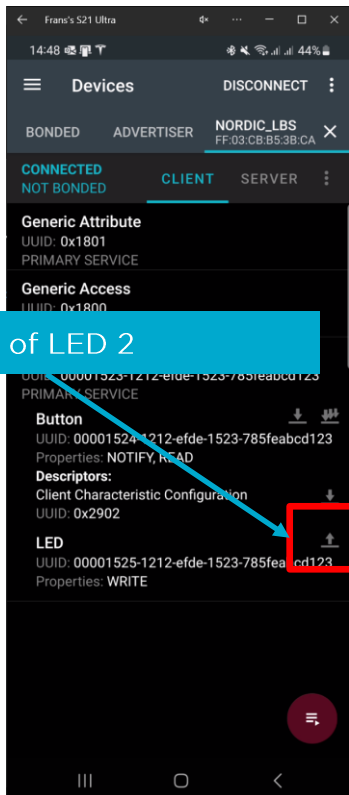
nRF Connect for Mobile



View button state advertisement



Change LED status on development kit



iOS users:

The Nordic LBS service, and specifically its LED state characteristic can be written with ByteArrays or Boolean values:

- 0x00 (off) / bool false
- 0x01 (on) / bool true

Analyze the application

Project configuration (file: prj.conf)

Enable Bluetooth

Setup as peripheral device

The name the devices will use during advertisement

```
8 CONFIG_BT=y
9 CONFIG_BT_PERIPHERAL=y
10 CONFIG_BT_DEVICE_NAME="Nordic_LBS"
```

Enable the Nordic Led Button Service

Enable the button feature

```
12 # Enable the LBS service
13 CONFIG_BT_LBS=y
14 CONFIG_BT_LBS_POLL_BUTTON=y
```

Enable the Development Kit library for adding support for the Leds and Buttons/switches

```
16 CONFIG_DK_LIBRARY=y
```

Scan Response packet

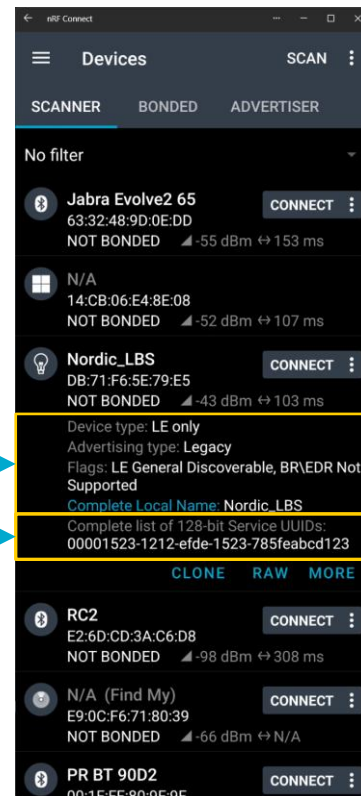
- There are numerous standard data types that can be included in the advertising data (either in the advertising packet or the scan response). These data types are defined in the Bluetooth Supplement to the Core Specification.
 - In this example we response with the UUID of the Nordic LBS service
 - Like with the advertisement package the Macro BT_DATA_BYTES is used.
-
- **BT_DATA_UUID128_ALL** Indicate the data type (Standard BLE UUID)
 - **BT_UUID_LBS_VAL** UUID for the Nordic LBS Service

```
48 static const struct bt_data sd[] = {  
49     BT_DATA_BYTES(BT_DATA_UUID128_ALL, BT_UUID_LBS_VAL),  
50 };  
51
```

Advertising & Response (Service UUID)

Advertising information

Response package with Service UUID



Define BLE call backs

Call back definition from HOST controller

*[connected, disconnected, security_changed,
le_param_req, le_param_updated, le_phy_updated,
le_data_len_updated, identity_resolved,
remote_info_available]*

```
92 ▾ BT_CONN_CB_DEFINE(conn_callbacks) = {  
93     .connected      = connected,  
94     .disconnected   = disconnected,  
95 ▾ #ifdef CONFIG_BT_LBS_SECURITY_ENABLED  
96     .security_changed = security_changed,  
97     #endif  
98 };
```

Call back definition for LBS Profile

```
163 ▾ static struct bt_lbs_cb lbs_callbacks = {  
164     .led_cb      = app_led_cb,  
165     .button_cb   = app_button_cb,  
166 };
```

Main function

Enable Bluetooth

Load paired devices information from flash when enable

Setup the callback programs for the BLE profile (led_cb & button_cb)

Start advertising using the define advertising package and scan response package.
BT_LE_ADV_CONN defines advertisement type.

```
223     err = bt_enable(NULL);
```

```
231     if (IS_ENABLED(CONFIG_SETTINGS)) {  
232         settings_load();  
233     }
```

```
235     err = bt_lbs_init(&lbs_callbacks);
```

```
241     err = bt_le_adv_start(BT_LE_ADV_CONN, ad, ARRAY_SIZE(ad),  
242                           sd, ARRAY_SIZE(sd));
```

Buttons and LED Library

Button and Led library for development kits

Define LEDs for BLE connection status

Blink interval

Define LED which is controlled by user

Define which button is used to indicate status

```
27 #include <dk_buttons_and_leds.h>
```

```
33 #define RUN_STATUS_LED      DK_LED1
34 #define CON_STATUS_LED      DK_LED2
35 #define RUN_LED_BLINK_INTERVAL 1000
36
37 #define USER_LED            DK_LED3
38
39 #define USER_BUTTON          DK_BTN1_MSK
```

Call back function on button event

```
168 static void button_changed(uint32_t button_state, uint32_t has_changed)
```

Define call back for button events

```
182 err = dk_buttons_init(button_changed);
```

Init I/O pins used by LED library

```
197 err = dk_leds_init();
```

Change Advertising

Change Scan Response packet

- The content of the scan response packet can be chosen. You're free to choose what to show. Let's change it to a website using a BT Uniform Resource Identifier (URI):

```
#define URL_STRING "//academy.nordicsemi.com"
static const struct {
    unsigned char prefix;
    const char url_string[sizeof(URL_STRING)];
} url_data = {
    .prefix = 0x17, // https
    .url_string = URL_STRING
};
static const struct bt_data sd[] = {
    BT_DATA(BT_DATA_URI, (const unsigned char *)&url_data, sizeof(url_data))
};
```

- `BT_DATA_URI` Indicate the data type (URI)
- `url_data[]` Text string with website / URL

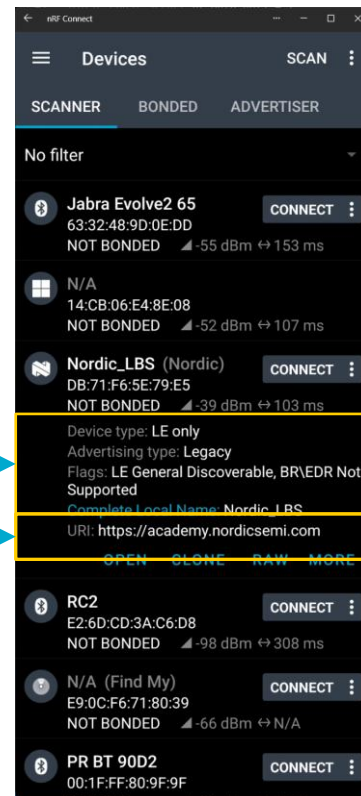
Advertising & Response (Service URI)

Note: When removing the “Service UUID” the nRF Blinky mobile app will not recognize the development board any longer,

Advertising information

Response package with URL string (URI Data)

Note: The Uniform Resource Identifier (URI) is NOT visible on iOS



Code challenge



LED0 stops blinking and is OFF when connected.



LED0 starts blinking when not connected and advertising

```
static bool app_connected_state;

static void connected(struct bt_conn *conn, uint8_t err)
{
    ....
    app_connected_state = true;
}

static void disconnected(struct bt_conn *conn, uint8_t reason)
{
    ....
    app_connected_state = false;
}

int main(void)
{
    ....
    for (;;) {
        if (app_connected_state) {
            dk_set_led_off(RUN_STATUS_LED);
        } else {
            dk_set_led(RUN_STATUS_LED, (++blink_status) % 2);
        }
        k_sleep(K_MSEC(RUN_LED_BLINK_INTERVAL));
    }
}
```

```
static bool app_disconnected = true;

static void connected(struct bt_conn *conn, uint8_t err)
{
    ....
    dk_set_led_off(RUN_STATUS_LED);
    app_disconnected = false;
}

static void disconnected(struct bt_conn *conn, uint8_t reason)
{
    ....
    app_disconnected = true;
}

int main(void)
{
    ....
    for (;;) {
        if (app_disconnected) {
            dk_set_led(RUN_STATUS_LED, (++blink_status) % 2);
        }
        k_sleep(K_MSEC(RUN_LED_BLINK_INTERVAL));
    }
}
```

Bluetooth LE Sniffer

DevAcademy - Lesson 6

DevAcadamy - Lesson 6 – BLE sniffer

- Communication is real-time making data exchange debugging difficult.
- Setting break points will break communication.
- Lower layers are difficult to monitor from the application.

- Objectives
 - Learn how to use a Bluetooth sniffer to capture advertising packets
 - Learn how to install and run Nordic's Bluetooth sniffer, nRF Sniffer, to capture and analyze Bluetooth LE packets
 - Through nRF Sniffer, learn how to follow a connection and inspect the communication
 - Observe and decrypt the Bluetooth LE communication of a secured connection
- <https://academy.nordicsemi.com/lessons/lesson-6-bluetooth-le-sniffer/>